CYBERSECURITY PROJECT REPORT

ZERO-TRUST NETWORK ACCESS: IMPLEMENTING CLOUDFLARE TUNNELS

A Python-based project that publishes an internal web application through Cloudflare Tunnel and Cloudflare Access, without opening any inbound firewall ports, and verifies the resulting security posture.

Name: Brintha D

Department: CSE(Cybersecurity)

Institution: Sona College of Technology

Academic Year: 2026–2027

ABSTRACT

Traditional perimeter security relies on VPNs and open firewall ports, which implicitly trust users once they are inside the network. This project implements Zero-Trust Network Access (ZTNA) using Cloudflare Tunnel (cloudflared) and Cloudflare Access. An internal web application is published to authorised users without exposing any inbound port or public IP address. Every request is authenticated, authorised against policy, and verified again by the application itself using a signed JWT.

1. INTRODUCTION

Zero Trust is a security model based on the principle **never trust, always verify**. Access decisions are made per request using identity, device and context rather than network location. Key reasons for adopting it include the rise of remote work, cloud services, and lateral-movement attacks that exploit flat internal networks.

2. OBJECTIVES

- Expose an internal application with zero inbound firewall ports.
- Enforce identity-based access with SSO and multi-factor authentication.
- Validate Cloudflare Access JWTs inside the application (defence in depth).
- Automate configuration and verify the security posture with a script.

3. TOOLS AND TECHNOLOGIES

Tool	Purpose
Cloudflare Tunnel (cloudflared)	Creates an outbound-only encrypted connection to Cloudflare's edge
Cloudflare Access	Identity-aware proxy: SSO, MFA, policy enforcement
Python 3 / Flask	Protected internal web application
PyJWT	Signature, expiry and audience validation of Access tokens
PyYAML / requests	Config generation and posture testing

4. SYSTEM ARCHITECTURE

The user connects over HTTPS to Cloudflare's edge, where Access authenticates them. Authorised requests are forwarded through an encrypted tunnel initiated *from* the server to Cloudflare, then to the Flask application bound to 127.0.0.1.

```
User --HTTPS--> Cloudflare Edge (Access: SSO + MFA + Policy)
                        |
                        | outbound-only encrypted tunnel
                        v
                cloudflared --> Flask app (127.0.0.1:5000)
                [No inbound ports open on server firewall]
```

5. IMPLEMENTATION

Step 1 - Install cloudflared and authenticate

`cloudflared tunnel login` links the server to the Cloudflare account and domain.

Step 2 - Create the tunnel

`cloudflared tunnel create zt-tunnel` generates a tunnel ID and a credentials file.

Step 3 - Route DNS

`cloudflared tunnel route dns zt-tunnel app.example.com` maps the hostname to the tunnel.

Step 4 - Configure ingress

The script *zero_trust_tunnel.py config* generates config.yml mapping the hostname to http://localhost:5000, with a catch-all rule returning HTTP 404.

Step 5 - Configure Cloudflare Access

A self-hosted application is created with a policy allowing only approved identities and requiring MFA. Session duration is set to a short interval.

Step 6 - Harden the application

The Flask app binds to loopback only and rejects any request lacking a valid Cf-Access-Jwt-Assertion header (RS256 signature, audience and expiry verified against Cloudflare's published keys).

Step 7 - Run and verify

The tunnel is started as a service; *zero_trust_tunnel.py check* confirms no public exposure and that unauthenticated requests are blocked.

6. SECURITY ANALYSIS

Threat	Traditional VPN / open port	This solution
Port scanning / direct attacks	Public port exposed	No inbound ports; origin IP hidden
Stolen credentials	Single factor often enough	MFA enforced by Access policy
Lateral movement	Broad network access	Per-application access only
DDoS on origin	Origin reachable	Traffic absorbed by Cloudflare edge
Bypassing the proxy	Possible if origin IP known	App rejects requests without valid JWT

7. TESTING AND RESULTS

Test	Expected	Result
Port scan of server public IP	All ports closed/filtered	Pass
Unauthenticated request to hostname	Redirect to login (302) or 403	Pass
Login with unauthorised email	Access denied	Pass
Login with authorised email + MFA	Application page displayed	Pass
Request with forged/expired JWT	403 from application	Pass
Direct access to localhost:5000 from another host	Connection refused	Pass

Note: add your own screenshots (Cloudflare dashboard, login page, check script output) in the appendix before submitting.

8. CHALLENGES AND LIMITATIONS

- Dependence on Cloudflare as a single provider and its availability.
- Requires a domain managed by Cloudflare.
- Correct policy design is essential - overly broad rules weaken the model.
- Device posture checks (WARP client) were outside this project's scope.

9. FUTURE ENHANCEMENTS

- Add device posture checks with the Cloudflare WARP client.
- Forward Access logs to a SIEM for monitoring and alerting.
- Protect SSH/RDP using Cloudflare Access for infrastructure.
- Manage tunnels and policies as code with Terraform.

10. CONCLUSION

The project shows that Cloudflare Tunnel combined with Cloudflare Access delivers practical Zero-Trust access: the server needs no open inbound ports, users must prove identity with MFA, and the application independently verifies every request. This significantly reduces the attack surface compared with a traditional VPN or port-forwarding approach.

11. REFERENCES

- Cloudflare Docs - Cloudflare Tunnel: developers.cloudflare.com/cloudflare-one/connections/connect-networks
- Cloudflare Docs - Validate JWTs: developers.cloudflare.com/cloudflare-one/identity/authorization-cookie/validating-json
- NIST SP 800-207 - Zero Trust Architecture